Advanced Artificial Intelligence and Machine Learning Lab 6

October 20, 2026

1 Lab 6

> Adapted from the Hugging Face LLM Course (`huggingface/course`, commit `8b893f0`, and its notebooks `huggingface/notebooks`, commit `3468b87`), Apache License 2.0: Chapter 7.3 "Fine-tuning a masked language model" (Exercises 1-3) and Chapters 3.2 and 3.4 "Processing the data" and "A full training loop" (Exercise 4). Code and prose are adapted and shortened. Changed from the source: a much smaller BERT (BERT-mini, 11M parameters, instead of DistilBERT/BERT-base) and smaller data subsets, so that every training run takes about 2-3 minutes on a CPU; the masking and the evaluation are written by hand instead of using library helpers; no Hugging Face Hub login or upload.

Session 8 showed how BERT is *pretrained* with masked language modeling (MLM) and then *fine-tuned* for a downstream task through its `[CLS]` token. This lab does both with a real pretrained BERT: you look at what it predicts for a `[MASK]`, build the MLM training data yourself, continue its pretraining on movie reviews (*domain adaptation*), and fine-tune it to detect paraphrases.

> **Instructor timing (2h session).** ~15 min: intro, setup and downloads (model ~45 MB, IMDB ~85 MB, MRPC ~1 MB). ~15 min: Exercise 1 - `[MASK]` predictions by hand. ~30 min: Exercise 2 - chunking and BERT's masking scheme. ~25 min: Exercise 3 - domain adaptation (a ~2.5 minute run). ~30 min: Exercise 4 - paraphrase detection (a ~2 minute run). ~5 min buffer.

1.1 Setup

These labs are designed to run on Google Colab - no local Python install needed. Open this notebook via Colab's GitHub loader (`colab.research.google.com/github/KarikS/lecture_advanced_ai_ml/blob/main/<path-to-this-notebook`) or File > Open notebook > GitHub tab, repo `KarikS/lecture_advanced_ai_ml`, then run the cell below once per session. Running the notebook locally instead (e.g. via the `exercises/python/` venv described in that folder's README) works unchanged - the cell below is a no-op there. No GPU is needed - each of the two training runs took 2-3 minutes with 4 CPU threads in testing. Colab already has `transformers` and `datasets` installed.

[1]:
```
import sys

IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
```

```
    import os
    if not os.path.exists('lecture_advanced_ai_ml'):
        !git clone https://github.com/KarikS/lecture_advanced_ai_ml.git
```

1.2 Imports

[2]:
```
import math
import time

import datasets
import torch
from datasets import load_dataset
from torch import Tensor
from torch.utils.data import DataLoader
from transformers import (AutoModelForMaskedLM,␣
 ↪AutoModelForSequenceClassification, AutoTokenizer,
                          DataCollatorForLanguageModeling,␣
 ↪DataCollatorWithPadding)
from transformers.utils import logging as transformers_logging

transformers_logging.set_verbosity_error()  # hide the expected "unused weights"␣
 ↪notes
transformers_logging.disable_progress_bar()
datasets.disable_progress_bars()
torch.manual_seed(0)
torch.set_num_threads(4)

CHECKPOINT = 'google/bert_uncased_L-4_H-256_A-4'
```

```
/tmp/claude-1000/-home-karik-lectures-lecture-i2dl-lecture-advanced-ai-
ml/715a1d6b-16e2-482b-b32c-52c99f701fd9/scratchpad/labenv/lib/python3.12/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

1.3 Exercise 1: What Does a Pretrained BERT Predict?

The source uses DistilBERT; we use **BERT-mini**, one of the small BERT models Google released alongside the original BERT: the same architecture and pretraining (MLM and next sentence prediction on English Wikipedia and BookCorpus), with 4 layers, $d_{\text{model}} = 256$ and 4 heads - 11M parameters instead of BERT-base's 110M. The checkpoint includes the pretrained MLM head, so we can ask it to fill in a `[MASK]` right away.

[3]:
```
tokenizer = AutoTokenizer.from_pretrained(CHECKPOINT)
mlm_model = AutoModelForMaskedLM.from_pretrained(CHECKPOINT)
print(f'{mlm_model.num_parameters() / 1e6:.1f}M parameters')
```

```
inputs = tokenizer('This is a great [MASK].', return_tensors='pt')
print(inputs['input_ids'])
print(tokenizer.convert_ids_to_tokens(inputs['input_ids'][0]))
```

```
11.2M parameters
tensor([[ 101, 2023, 2003, 1037, 2307,  103, 1012,  102]])
['[CLS]', 'this', 'is', 'a', 'great', '[MASK]', '.', '[SEP]']
```

The tokenizer has added `[CLS]` at the start and `[SEP]` at the end. The model returns one vector of logits over the 30,522-token vocabulary for *every* position; to fill in the mask, we need the logits at the `[MASK]` position.

[4]:
```
def top_mask_predictions(model, text: str, k: int = 5) -> list[str]:
    inputs = tokenizer(text, return_tensors='pt')
    with torch.no_grad():
        logits = model(**inputs).logits  # (1, seq_len, vocab_size)
# TODO: Find the position of the [MASK] token (tokenizer.mask_token_id) in
# inputs['input_ids'], take the logits at that position, and return the k tokens
# with the highest logits as strings (torch.topk, tokenizer.decode([token_id])).


PROBE_SENTENCES = ['This is a great [MASK].',
                   'I went to the [MASK] to buy some bread.',
                   'The actor gave a [MASK] performance.']
for sentence in PROBE_SENTENCES:
    print(f'{sentence:45s} {top_mask_predictions(mlm_model, sentence)}')
```

Question: As humans, we can think of many completions for "This is a great [MASK]." - "day", "ride", "painting". What decides which ones the model predicts?

1.4 Exercise 2: Preparing the Data for Masked Language Modeling

To adapt the model to movie reviews, we use the IMDb dataset (Large Movie Review Dataset), a corpus of highly polarized reviews usually used for sentiment analysis. We don't need its labels for language modeling. The source trains on 10,000 chunks; we take 2,000 reviews. The training split is sorted by label, so we shuffle before selecting.

[5]:
```
imdb = load_dataset('stanfordnlp/imdb', split='train').shuffle(seed=42).
↪select(range(2000))
print(imdb)
print(imdb[0]['text'][:300])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
```

```
Dataset({
    features: ['text', 'label'],
    num_rows: 2000
})
```

```
There is no relation at all between Fortier and Profiler but the fact that both
are police series about violent crimes. Profiler looks crispy, Fortier looks
classic. Profiler plots are quite simple. Fortier's plot are far more
complicated... Fortier looks more like Prime Suspect, if we have to spot
```

For both autoregressive and masked language modeling, a common preprocessing step is to **concatenate all examples and split the whole corpus into chunks of equal size**, instead of tokenizing each example on its own: individual examples would otherwise be truncated if they are too long, losing text that is useful for language modeling, or padded if they are short. So we tokenize *without* truncation, then group the tokens into chunks of 128 (BERT's context is 512 tokens, but shorter chunks train faster). The last, incomplete chunk is dropped. For MLM, the labels start out as a copy of the inputs - the masking happens later.

[6]:
```
CHUNK_SIZE = 128

tokenized_imdb = imdb.map(lambda batch: tokenizer(batch['text']), batched=True,
                          remove_columns=['text', 'label'])
print(tokenized_imdb)
print('first review lengths:', [len(ids) for ids in tokenized_imdb[:
 ↪5]['input_ids']])
```

```
Dataset({
    features: ['input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 2000
})
first review lengths: [179, 159, 237, 163, 936]
```

[7]:
```
def group_texts(examples: dict[str, list[list[int]]]) -> dict[str,␣
 ↪list[list[int]]]:
# TODO: `examples` maps each feature name (input_ids, token_type_ids,␣
 ↪attention_mask)
# to a list of tokenized reviews. For every feature:
# 1. concatenate all reviews into one long list,
# 2. cut it into chunks of CHUNK_SIZE, dropping the last incomplete chunk.
# Return a dict with the chunked features plus a 'labels' entry that copies the
# chunked input_ids.


lm_dataset = tokenized_imdb.map(group_texts, batched=True).
 ↪train_test_split(test_size=0.1, seed=42)
lm_dataset.set_format('torch')
print(lm_dataset)
assert all(len(ids) == CHUNK_SIZE for ids in lm_dataset['train'][:
 ↪100]['input_ids'])
print(tokenizer.decode(lm_dataset['train'][1]['input_ids']))
```

Grouping produced many more examples than the 2,000 reviews, and a chunk can span the end of

one review and the start of the next - look for the `[SEP]` and `[CLS]` tokens in the middle of the decoded chunk above.

BERT's masking scheme (Session 8). The labels are still identical to the inputs - we are missing the key step: inserting `[MASK]` tokens at random positions. BERT selects 15% of the tokens as prediction targets, and of those replaces 80% by `[MASK]`, 10% by a random token, and leaves 10% unchanged. Only the selected tokens count in the loss: all other labels are set to `-100`, the index that PyTorch's cross-entropy (and so the model's loss) ignores. Special tokens (`[CLS]`, `[SEP]`) are never selected.

[8]:
```
def mask_tokens(input_ids: Tensor, mlm_probability: float = 0.15,
                generator: torch.Generator | None = None) -> tuple[Tensor,␣
↪Tensor]:
    """BERT-style masking for a batch of token IDs. Returns (masked inputs,␣
↪labels)."""
    labels = input_ids.clone()
    inputs = input_ids.clone()
    special = torch.tensor([tokenizer.get_special_tokens_mask(ids,␣
↪already_has_special_tokens=True)
                            for ids in input_ids.tolist()], dtype=torch.bool)
# TODO:
# 1. Select each non-special token with probability mlm_probability
# (torch.bernoulli on a probability tensor, with generator=generator).
# 2. Set the labels of all non-selected tokens to -100.
# 3. Of the selected tokens, replace 80% by tokenizer.mask_token_id, then half␣
↪of the
# remaining selected ones (10% overall) by a random token ID
# (torch.randint(len(tokenizer), ...)); the rest stay unchanged.
    return inputs, labels
```

[9]:
```
def masking_statistics(inputs: Tensor, labels: Tensor, original: Tensor) -> str:
    selected = labels != -100
    return ('selected {:.1%} of all tokens; of those: [MASK] {:.1%}, random {:.
↪1%}, unchanged {:.1%}'
            .format(float(selected.float().mean()),
                    float((inputs[selected] == tokenizer.mask_token_id).float().
↪mean()),
                    float(((inputs[selected] != tokenizer.mask_token_id)
                           & (inputs[selected] != original[selected])).float().
↪mean()),
                    float((inputs[selected] == original[selected]).float().
↪mean())))


sample_ids = lm_dataset['train'][:256]['input_ids']
```

```
masked, labels = mask_tokens(sample_ids, generator=torch.Generator().
 ↪manual_seed(0))
print('ours:        ', masking_statistics(masked, labels, sample_ids))

# Hugging Face's own collator implements the same scheme - the source course␣
 ↪uses it directly.
hf_collator = DataCollatorForLanguageModeling(tokenizer, mlm_probability=0.15)
hf_batch = hf_collator([{'input_ids': ids} for ids in sample_ids])
print('Hugging Face:', masking_statistics(hf_batch['input_ids'],␣
 ↪hf_batch['labels'], sample_ids))

print(tokenizer.decode(masked[0][:60]))
```

Both should select about 15% of the tokens (slightly less, because special tokens are never selected), with about 80% / 10% / 10% among those. A random token can by chance be the original one, so "unchanged" comes out a little above 10%.

Question: Why doesn't BERT simply replace *all* selected tokens by `[MASK]`?

1.5 Exercise 3: Domain Adaptation - Continuing BERT's Pretraining on Movie Reviews

Fine-tuning a pretrained language model on in-domain text before training a task-specific head is called **domain adaptation**; it was popularized by ULMFiT in 2018. Its loss is the MLM loss on the masked positions, and we report its exponential, the **perplexity**: a model that is not "perplexed" by unseen text assigns high probability to the right tokens.

Since masking is random, the validation perplexity would change from one evaluation to the next. The source course's solution: mask the validation set *once*, with a fixed seed, and reuse it. We also use 256 validation chunks to keep evaluation fast.

[10]:
```
val_ids = lm_dataset['test'][:256]['input_ids']
val_inputs, val_labels = mask_tokens(val_ids, generator=torch.Generator().
 ↪manual_seed(1234))
val_batches = [(val_inputs[i:i + 64], val_labels[i:i + 64]) for i in range(0,␣
 ↪len(val_ids), 64)]


def mlm_perplexity(model) -> float:
    model.eval()
    with torch.no_grad():
        losses = [float(model(input_ids=x, labels=y).loss) for x, y in␣
 ↪val_batches]
    model.train()
    return math.exp(sum(losses) / len(losses))
```

```
print(f'perplexity before domain adaptation: {mlm_perplexity(mlm_model):.2f}')
```

The training loop is the same as for any PyTorch model - Hugging Face models compute their own loss when `labels` are passed. New masks are drawn for every batch, so the model never sees the same masking of a chunk twice. We train for 60 steps of 32 chunks (about 1,900 of the ~3,900 training chunks), with AdamW and a learning rate of 10^{-4} (the source: $5 \cdot 10^{-5}$ for a much longer run).

[11]:
```
MLM_STEPS, MLM_BATCH, MLM_LR = 60, 32, 1e-4

mask_generator = torch.Generator().manual_seed(0)
train_loader = DataLoader(lm_dataset['train']['input_ids'],␣
 ↪batch_size=MLM_BATCH, shuffle=True,
                          generator=torch.Generator().manual_seed(0))
optimizer = torch.optim.AdamW(mlm_model.parameters(), lr=MLM_LR)

mlm_model.train()
t0 = time.time()
for step, batch_ids in enumerate(train_loader, start=1):
# TODO: Mask the batch with your mask_tokens (generator=mask_generator), compute
# the model's loss (model(input_ids=..., labels=...).loss), backward pass,␣
 ↪optimizer
# step, zero the gradients. Call the loss `loss`.
    if step % 10 == 0:
        print(f'STEP: {step:3d}\tTRAIN LOSS: {loss.item():.3f}\tELAPSED: {time.
 ↪time() - t0:.0f}s')
    if step == MLM_STEPS:
        break

print(f'perplexity after domain adaptation: {mlm_perplexity(mlm_model):.2f}')
for sentence in PROBE_SENTENCES:
    print(f'{sentence:45s} {top_mask_predictions(mlm_model, sentence)}')
```

For reference (seeds 0, 1, 2, one CPU run each): perplexity drops from 21.26 to 18.9-19.3 in about 2.5 minutes of training. "This is a great [MASK]." changes from *success, deal, bird, flood, legend* to answers such as *film, movie, story* - but also *mistake* and *disaster*: the model has learned what IMDb reviews talk about, not whether they are positive.

Question: Lab 7 reports a perplexity for a GPT-style model too. Why can't you compare the two numbers?

1.6 Exercise 4: Fine-Tuning BERT for Paraphrase Detection

Now the second half of Session 8: fine-tuning. We use **MRPC** (Microsoft Research Paraphrase Corpus, part of the GLUE benchmark): pairs of sentences with a label saying whether they mean the same thing. It is small - 3,668 training pairs and 408 validation pairs - so it trains quickly. We

start again from the original pretrained checkpoint, not the domain-adapted one: MRPC consists of news sentences, not movie reviews.

[12]:
```
mrpc = load_dataset('nyu-mll/glue', 'mrpc')
print(mrpc)
print(mrpc['train'].features['label'])
print(mrpc['train'][0])
```

```
DatasetDict({
    train: Dataset({
        features: ['sentence1', 'sentence2', 'label', 'idx'],
        num_rows: 3668
    })
    validation: Dataset({
        features: ['sentence1', 'sentence2', 'label', 'idx'],
        num_rows: 408
    })
    test: Dataset({
        features: ['sentence1', 'sentence2', 'label', 'idx'],
        num_rows: 1725
    })
})
ClassLabel(names=['not_equivalent', 'equivalent'])
{'sentence1': 'Amrozi accused his brother , whom he called " the witness " , of
deliberately distorting his evidence .', 'sentence2': 'Referring to him as only
" the witness " , Amrozi accused his brother of deliberately distorting his
evidence .', 'label': 1, 'idx': 0}
```

We can't pass two separate sentences to the model - they have to be handled as a pair. Given two texts, the tokenizer builds the input BERT expects, `[CLS] sentence1 [SEP] sentence2 [SEP]`, and the `token_type_ids` that tell the model which part is the first sentence (type 0) and which is the second (type 1) - BERT learned these segment embeddings during pretraining, through next sentence prediction (Session 8).

[13]:
```
pair = tokenizer(mrpc['train'][0]['sentence1'], mrpc['train'][0]['sentence2'])
for token, token_type in list(zip(tokenizer.
 ↪convert_ids_to_tokens(pair['input_ids']), pair['token_type_ids']))[:40]:
    print(f'{token}:{token_type}', end='  ')
```

```
[CLS]:0  am:0  ##ro:0  ##zi:0  accused:0  his:0  brother:0  ,:0  whom:0  he:0
called:0  ":0  the:0  witness:0  ":0  ,:0  of:0  deliberately:0  di:0  ##stor:0
##ting:0  his:0  evidence:0  .:0  [SEP]:0  referring:1  to:1  him:1  as:1
only:1  ":1  the:1  witness:1  ":1  ,:1  am:1  ##ro:1  ##zi:1  accused:1  his:1
```

We tokenize every pair (no padding yet), remove the text columns and rename `label` to `labels`, the argument name the model expects. Padding happens per batch: **dynamic padding** pads each batch only to its own longest pair, instead of padding everything to the longest pair in the dataset. `DataCollatorWithPadding` does this.

[14]:
```
def tokenize_pairs(batch: dict) -> dict:
# TODO: Tokenize batch['sentence1'] and batch['sentence2'] as pairs, with
# truncation=True.


mrpc_tokenized = (mrpc.map(tokenize_pairs, batched=True)
                  .remove_columns(['sentence1', 'sentence2', 'idx'])
                  .rename_column('label', 'labels'))
padding_collator = DataCollatorWithPadding(tokenizer)
cls_train_loader = DataLoader(mrpc_tokenized['train'], batch_size=16,␣
↪shuffle=True,
                              collate_fn=padding_collator, generator=torch.
↪Generator().manual_seed(0))
cls_val_loader = DataLoader(mrpc_tokenized['validation'], batch_size=64,␣
↪collate_fn=padding_collator)

batch = next(iter(cls_train_loader))
print({k: tuple(v.shape) for k, v in batch.items()})
```

`AutoModelForSequenceClassification` puts a new, randomly initialized classification head on top of BERT's `[CLS]` representation - the fine-tuning setup from Session 8. The two possible errors of a paraphrase detector matter differently depending on the use, and the classes are imbalanced, so GLUE reports **F1** for MRPC alongside accuracy: $F_1 = \frac{2TP}{2TP+FP+FN}$, with "paraphrase" as the positive class.

[15]:
```
def evaluate_classifier(model) -> tuple[float, float]:
    model.eval()
    n_correct = n_total = tp = fp = fn = 0
    with torch.no_grad():
        for batch in cls_val_loader:
# TODO: Predict the class of every pair (argmax over model(**batch).logits)
# and update n_correct, n_total, tp, fp, fn (positive class: label 1).
    model.train()
    return n_correct / n_total, 2 * tp / (2 * tp + fp + fn)


majority = sum(mrpc['validation']['label']) / len(mrpc['validation'])
print(f'majority baseline (always "paraphrase"): accuracy {majority:.3f}, F1 {2␣
↪* majority / (1 + majority):.3f}')

torch.manual_seed(0)
cls_model = AutoModelForSequenceClassification.from_pretrained(CHECKPOINT,␣
↪num_labels=2)
print('before fine-tuning: accuracy {:.3f}, F1 {:.3f}'.
↪format(*evaluate_classifier(cls_model)))
```

The training loop follows the source: AdamW, and a learning rate that decays linearly from its maximum to 0 over all training steps (the `Trainer`'s default). The source trains BERT-base for 3 epochs at $5 \cdot 10^{-5}$; for our much smaller model, 2 epochs at 10^{-4} worked better in testing.

[16]:
```
CLS_EPOCHS, CLS_LR = 2, 1e-4

optimizer = torch.optim.AdamW(cls_model.parameters(), lr=CLS_LR)
total_steps = CLS_EPOCHS * len(cls_train_loader)
scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lambda step: 1 - step /␣
 ↪total_steps)

cls_model.train()
t0 = time.time()
for epoch in range(1, CLS_EPOCHS + 1):
    for batch in cls_train_loader:
# TODO: One training step: forward pass (the model returns .loss when the batch
# contains labels), backward pass, optimizer step, scheduler step, zero the
# gradients.
    accuracy, f1 = evaluate_classifier(cls_model)
    print(f'EPOCH: {epoch}\tVAL ACCURACY: {accuracy:.3f}\tVAL F1: {f1:.
 ↪3f}\tELAPSED: {time.time() - t0:.0f}s')
```

For reference (seeds 0, 1, 2): before fine-tuning, accuracy is 0.42-0.60 and F1 is 0.41-0.73. After two epochs (about 2-2.5 minutes on CPU), accuracy is 0.77-0.79 and F1 is 0.84-0.85. Always answering "paraphrase" gets accuracy 0.684 and F1 0.812.

Question: Before fine-tuning, the classifier is *worse* than always answering "paraphrase". Why - BERT is pretrained, after all?

Question: Why report F1 next to accuracy for MRPC? Look at the majority baseline.

1.7 Conclusion

Here's what you should take away from this lab:

- A pretrained masked language model fills in a `[MASK]` from the logits at that position (Exercise 1), and its predictions reflect the corpus it was pretrained on.
- BERT's training data is simple to build: concatenate the corpus, cut it into chunks, and select 15% of the tokens with the 80/10/10 replacement scheme (Exercise 2).
- Continuing the MLM training on in-domain text (domain adaptation) shifts the model's predictions towards that domain within a few minutes (Exercise 3).
- Fine-tuning puts a new head on the `[CLS]` token and trains everything for the task; judge the result against a baseline, with metrics that suit the class balance (Exercise 4).